



Running Compose on Incus

The Story, The Migration, and What Changes Under the Hood

Speaker: René Jochum

Linux Containers (**lxc**)



Executive Summary

You can run standard `compose.yaml` files natively on Incus without Docker

- **The Situation:** Developers love Compose; sysadmins love Incus for security, storage, and networking
- **The Complication:** Docker-in-LXC is a clunky workaround (two runtimes, firewall collisions, layered filesystems)
- **The Question:** How do we run unchanged Compose stacks natively on Incus?
- **The Answer:** `incus-compose`
 - **Drop-in:** Same `compose.yaml`, parsed with official `compose-go`
 - **Remote-First:** Drive remote servers over HTTPS from macOS, Windows, or Linux with zero local VMs
 - **Escape Hatch:** Where Compose ends, the full Incus API stands ready via `x-incus`



Situation: A 10-Year Love Affair With Incus

- **Since 2015 (LXD Days):** A single compact daemon with a clean REST API
- **Three Workloads Unified in One Engine:**
 - **LXC System Containers:** Full Linux OS with systemd in 200ms
 - **KVM Virtual Machines:** Real hardware virtualization under the exact same CLI
 - **OCI App Containers (2024):** Direct Docker/OCI image execution without Docker
- **Real Infrastructure Primitives:** Unprivileged user namespaces, native ZFS/Btrfs CoW, and OVN
- **Community-Driven:** Forked to Incus under `linuxcontainers.org` with first-class multi-distro packages



Complication: The "Compose Wall" & The Workarounds

Developers publish applications as `compose.yaml`, not infrastructure scripts:

- **The Clunky Workarounds:**

- **Side-by-side on the host:** Docker injects raw `iptables` rules that collide directly with Incus's managed NAT and ACL rules
- **Docker inside LXC:** Putting an engine inside an engine (nested namespaces, overlays, concessions)
- **Dedicated VM:** Avoids firewall collision, but introduces heavy hypervisor memory and CPU overhead

- **The Native Gap:**

- Incus added native OCI container support in 2024
- But Incus is an infrastructure manager (`incus launch`), not a stack manager
- Writing a 20-step bash script to launch a 5-tier app is tedious and brittle



The Spark: Christmas 2025 & The Fork

- **Late 2025:** Discovered Brian Ketelsen's archived prototype while building `octocompose`
- **The Fork:** Rebuilt from scratch over Christmas 2025 to create a true production engine
- **Christmas Eve 2025 (Beta 1):** Brian Ketelsen passed the torch on the Linux Containers forum
- **Brian's Prophecy:** *"I stopped because real health-check orchestration was too much work without AI... hope you carry the torch!"*
- **800 Hours Tracked in Kimai:** Evolving a holiday project into `lxc/incus-compose` v1.3 through relentless dogfooding



The Architecture: Three Rewrites

- **Attempt 1: The Monolith**
 - Giant client struct (`EnsureService`, `EnsureNetwork`) — functional, but coupled
- **Attempt 2: Priorities & KISS**
 - Numeric sorting (profiles 512, images 1024, instances 8192) — simple, but leaked state
- **Attempt 3: Boring Architecture (Today)**
 - Uniform `Resource` interface with priority-ordered execution
 - Composable Pre/Post Hooks for dry-run, logging, and error enrichment



From Fork to Successor: 22 Betas & Production Dogfooding

A tool is only as good as the systems that depend on it:

- **22 Betas Across Two Months:** Iterating in public with the Incus community
- **Brian Ketelsen Reappeared:** The prototype's author showed up hours before release: *"It looks like you've solved all of the really big problems"*
- **Pulled into Linux Containers:** Stéphane Graber brought it into `lxc/incus-compose`
- **Tested in Daily Production (The Dogfooding Loop):**
 - Immich (photos), Wiki.js & LeafWiki (docs), Gitea (code)
 - Caddy (ingress), Kimai (tracking hours on incus-compose itself), split-horizon DNS
 - *"When a release broke something, my photos stopped syncing. That is the test suite that matters"*



75% Test Coverage: When Real Life Fights the Test Suite

- **Zero Mocks:** Real reliability demands testing against real Incus daemons and storage
- **Immutable Regressions:** Every bug across 22 betas became an automated integration test
- **Battle 1 (Rapid-Fire Nginx):** Hardened fast startup/teardown races and socket binding conflicts
- **Battle 2 (`fsync` Storms):** Massive parallel disk cycles choked Incus — forced concurrency limits
- **The Outcome:** 75% coverage driving real container lifecycles from start to finish



The Remote-First Advantage

```
[ Your Laptop (macOS / Linux / Windows) ] --- HTTPS / TLS ---> [ Remote Incus Host or Cluster ]  
incus-compose CLI (static binary)                                incusd (OCI Stacks, LXC, VMs)
```

- **Zero Hypervisor Tax:** No Docker Desktop, no WSL2, and zero battery drain on your laptop
- **True Client-Server:** Drive remote production servers or clusters directly over HTTPS
- **Unified Control:** One static binary orchestrates remote instances, storage, and networks



Migrating a Project: Zero Translation

Migration requires **zero YAML translation**:

```
# In your existing project directory:  
incus-compose up -d
```

- **The Drop-In Contract:** If it works in Docker Compose and fails here, that is a bug in `incus-compose`
- **Official Parser:** Uses `compose-go` (the same library Docker Compose uses)
- **Automatic Resolution:** Evaluates `.env` files, variable interpolation, and dependency DAG
- **Resource Generation:**
 - Creates an isolated Incus project for the stack
 - Creates managed bridge networks and custom storage volumes
 - Pulls OCI images and launches unprivileged containers in parallel



What Changes: 1. Clean Networking & Native ACLs

- **Docker's Chaos:**

- Manipulates raw `iptables` / `nftables` behind the scenes
- Bypasses host firewalls (like UFW) and collides with Incus routing

- **Incus-Compose Cleanliness:**

- Stacks run on an isolated Incus managed bridge (`icompose0`) or OVN network
- Respects Incus's **native NAT and network ACL rules** instead of fighting them
- **Real IP Addresses:** Every container gets its own IP address on the internal bridge
- Port publishing uses clean Incus proxy devices or kernel NAT



What Changes: 2. Storage & The Volume Seeding Trap

- **Native Storage Pools:** Volumes are real Incus storage volumes on ZFS, Btrfs, LVM, or Linstor (DRBD)
- **The Volume Seeding Trap (Solved in v1.3):**
 - Docker auto-populates empty volumes with image defaults (e.g. Nginx `/etc/nginx/conf.d`)
 - Native Incus volumes mounted empty, causing default image configs to fail
 - `incus-compose` inspects and seeds initial image files into new volumes automatically
- **Shifted Permissions:** Unprivileged UID/GID mappings handled seamlessly by the hypervisor



What Changes: 3. Health Checks & `ic-healthd`

- **The Native Gap:** Incus has no native container healthcheck or restart supervisor
- **Evolution to Shared Host Daemon:**
 - **v1.0 (Sidecars):** Per-project sidecars consumed user container quotas
 - **v1.2 (Host Daemon):** Collapsed into one shared daemon in the `incus-compose` project
 - **Efficiency:** A new stack costs a goroutine and a map entry, not a container
- **Event-Driven Execution:**
 - Subscribes to the Incus WebSocket (`/1.0/events`) with zero polling overhead
 - Executes `test:` commands, handles restarts, and unblocks `service_healthy` dependencies



What Changes: 4. Seamless Local OCI Builds

Building local images on Incus alone is an exhausting chore:

- **Without incus-compose:** Build with Docker/Podman → export tarball → run `incus image import` → tag alias → relaunch
- **With `incus-compose build`:**

```
services:  
  web:  
    build: .
```

```
incus-compose up --build
```

- Automatically detects your local builder: `buildah`, `podman`, or `docker`
- Builds rootfs and streams it directly into the Incus project over the REST API
- Automatically handles image caching and instance recreation



The Escape Hatch: Never Weld It Shut

- **The Beta 19 Frigate Breakthrough (@blurry):**
 - Needed CIFS host mounts with custom `uid` / `gid` unsupported by the Compose spec
 - `x-incus` attached the mount directly via native Incus disk device syntax
 - Result: Full production migration achieved with zero engine code changes
- **Companion Files (`compose.incus.yaml`):**
 - Keep upstream `compose.yaml` untouched; declare hypervisor tuning in the companion
 - Storage pool routing (bulk ZFS HDD for media, fast NVMe for databases)
 - Direct hardware passthrough for NVIDIA/AMD GPUs, USB devices, and custom ACLs



Where the Limits Are: Hard Lessons & Upstream Fixes

- **Stop Timeouts:** Enforced explicit client-side timeouts to guarantee graceful shutdown
- **Entrypoint vs. Command:** Embedded an OCI registry client to inspect manifests and preserve Compose semantics
- **Upstream Incus Fixes (`lxc/incus#3785`):** Patched event races upstream to honor the drop-in contract
- **Concurrency Fork (`iclient`):** Built thread-safe isolated event connections for parallel worker pools
- **Dockerfile Healthchecks:** Hypervisors discard image metadata — declare `healthcheck:` in `compose.yaml`
- **Scope Boundary:** Built for single servers, edge nodes, and small clusters (not Kubernetes or Swarm)



Superpowers You Didn't Have in Docker

1. Stack-Aware Atomic Backups

```
incus-compose backup create      # Snapshots all volumes into <project>-backup
incus-compose backup list
incus-compose backup restore <id> # Instant rollback; immune to `down -v`
```

2. Private Port Tunneling

```
incus-compose port-forward db 5432 # Tunnel local port over Incus HTTPS
```

- **Backups live in an isolated Incus project:** Protected from accidental deletion
- **Private Port-Forward:** Inspect internal databases without exposing ports to the host or internet
- **Air-Gapped Ready:** Two-stage cache; `--pull never` guarantees zero registry calls



Where Things Go: The Expanding Ecosystem

- **Dynamic Ingress:** `incus-caddy-config`
 - Zero-touch reverse proxy for Caddy on Incus driven by `ievent`
 - Stages Caddyfiles via SFTP directly to the storage volume (`/config`), formats, and reloads
 - Auto-discovers and load-balances instances via labels (`edge: "photos.example.com, upstream=2283"`)
- **Split-Horizon DNS:** `ic-dns` (in `develop`)
 - Integrated authoritative DNS server (`incus-compose dns {up|down|status|logs}`)
- **Clustering & An Operator? (Under Consideration)**
 - Containers stay pinned to nodes for data locality and stability
 - Exploring declarative operator reconciliation for boot-time ordering



Standing on Shoulders: Community & Thanks

- **The Foundation:** Brian Ketelsen (@bketelsen), Stéphane Graber (@stgraber)
- **Community Heroes:**
 - @blurry: Masterclass bug reports (Frigate, CIFS mounts, GPU/USB passthrough)
 - @pyrodogg: Service DNS, seeding critique & the milestone: *"It just works"*
- **Contributors & Testers:**
 - @neitsab (AUR), @ishaan-jindal (backups/NAT), @alien43, @Sagi, @tofil, @kgoetz, @edorgeville, @bburky
 - Deep private debug sessions testing proprietary enterprise stacks
- **The Open Source Core:** `compose-go`, `lxc/incus`, `dominikbraun/graph`, `go-selfupdate`, `cupaloy`



Summary: Boring Architecture Wins

- **Drop-in:** Run unchanged `compose.yaml` stacks with native ZFS, Btrfs, and OVN
- **Remote-First:** Drive remote servers over HTTPS with zero local VM overhead
- **Escape Hatches:** Full hypervisor power unlocked via `x-incus` and companion files
- **Production-Hardened:** Immich, Wiki.js, Gitea, Caddy, and Kimai running 24/7

```
curl -sSfL https://raw.githubusercontent.com/lxc/incus-compose/main/install.sh | sh -s -- -b ~/.local/bin
```

- **Links:** incus-compose.org · github.com/lxc/incus-compose · discuss.linuxcontainers.org



Appendix: AI as a Sparring Partner

How AI pair-programming actually worked across 800 hours:

- **Not an Autopilot:** It does not "write software for you"
- **A Relentless Sparring Partner:**
 - Started with Claude, switched to Gemini for friendly, iterative collaboration
 - "The Boring One" asking what others made when passing between agents: *"What is this for?"*
 - Challenges over-abstraction and forces helpers to justify their existence
- **Test Generation & Edge Cases:**
 - Synthesizing edge-case integration tests for complex lifecycle transitions
 - Verifying failure modes, signal handling, and cleanup guarantees
- **Strict Discipline:** Human architectural direction; AI verification and review loops